# No-Code Neural Networks and the Limits of Keyword Matching: What a Sports E-Commerce Sentiment Classifier Taught Me About Tagged Learning

Luke Joaquin Bernardo
*College of Computer Studies and Engineering*
*Jose Rizal University*
Taguig City, Philippines
lukejoaquin.bernardo@my.jru.edu

***Abstract*—This report walks through what I found while building a no-code text classifier using Machine Learning for Kids. I trained a neural network to sort customer messages from a sports team e-commerce and ticketing platform into three categories: Positive_Praise, Negative_Complaint, and Urgent_Support. I used 100 tagged examples per class, tested the model on clear phrases, sarcastic or ambiguous phrases, and out-of-vocabulary questions, then went back and added targeted training examples to fix what broke. The model scored 90% on clear, everyday phrasing but dropped to about 33% on sarcasm and ambiguous wording, which lines up with what the exercise predicted. I use these results to answer three questions: why hardcoded IF/ELSE rules do not scale for this kind of problem, why dataset size and diversity matter so much for accuracy, and where in the pipeline feature extraction actually happens.**

## I. Introduction

For this exercise I built a customer feedback classifier for a sports team's online store and ticketing app. I picked this domain because it naturally produces all three kinds of messages the exercise asks for: fans praising a smooth checkout, fans complaining about a damaged jersey, and fans in a panic because their tickets will not load before kickoff. I wrote 100 example sentences per class, so 300 sentences total, then trained the model in Machine Learning for Kids and tested it in stages. The rest of this report goes through what I saw at each stage and what I think it means.

## II. Methodology

I set up a Recognizing Text project and created three classes: Positive_Praise, Negative_Complaint, and Urgent_Support. Instead of the minimum 10 examples per class, I wrote 100 for each one, aiming for a mix of everyday phrasing rather than repeating the same sentence structure over and over. Once training finished, I tested the model in three rounds, following the structure the exercise laid out.

Test A used 30 clear, unambiguous phrases, 10 per class, that were never part of training. Test B used 30 phrases built around sarcasm, sentiment reversal, and hedged urgency, again 10 per class. Test C used 30 out-of-vocabulary questions, mostly general account or billing questions that do not belong cleanly to any one class. For each test I recorded the predicted category, the confidence score, and whether the prediction matched what I expected. Then, in a refinement step, I added 32 new training examples that specifically targeted the failure patterns from Test B, retrained the model, and re-tested seven of the phrases that failed the first time.

TABLE I
Overall Accuracy by Test Round

| Test Round | Total Tested | Correct | Accuracy |
|---|---|---|---|
| A: Clear Patterns | 30 | 27 | 90.0% |
| B: Ambiguous / Sarcastic | 30 | 10 | 33.3% |
| C: Out-of-Vocabulary | 30 | N/A | no fixed label |

## III. Results

Table I shows the overall accuracy for each test round. The pattern is exactly what the exercise description warned about: the model is strong on plain language and weak on anything that needs context to read correctly.

In Test A, the model missed only three phrases out of thirty, and all three misses came from Urgent_Support, which sometimes got read as Negative_Complaint. That makes sense to me, since an urgent message and a complaint often share a lot of the same words, like "cancelled" or "mistake." The gap is really about tone and time pressure, not vocabulary, and tone is harder for the model to pick up from a short sentence.

Test B is where things fell apart. Table II breaks down accuracy per class. Positive_Praise sentences that started negative and flipped positive, like "I was ready to cancel my membership, but this renewal experience actually changed my mind," were read as Negative_Complaint almost every time. Negative_Complaint sentences that opened with a sarcastic positive word, like "Sure, love it when my saved cart empties itself right before checkout," were flagged as Positive_Praise, and often with high confidence, which is worse than a low-confidence miss because the model looked

TABLE II
TEST B ACCURACY BY CLASS (BEFORE REFINEMENT)

| Class | Correct/10 | Accuracy |
|---|---|---|
| Positive_Praise | 4 | 40.0% |
| Negative_Complaint | 2 | 20.0% |
| Urgent_Support | 4 | 40.0% |

TABLE III
REFINEMENT RE-TEST OUTCOMES (7 PHRASES)

| Outcome | Count |
|---|---|
| Improved (wrong → correct) | 3 |
| Still incorrect | 4 |
| Unchanged and already correct | 0 |

sure of something wrong. Urgent_Support phrases that hedged the urgency, like "Not trying to overreact, but the whole ticketing system has been down for an hour," mostly got labeled Negative_Complaint instead.

Test C did not have a single right answer for each phrase, since these were general questions like "Can I customize the notifications for just my favorite players?" that do not obviously belong to any one class. What stood out to me here was how the confidence scores split. Most of the time the model landed somewhere in the 10 to 30 percent range, which is a reasonable way for it to say "I am not sure." But a few phrases, like "How do I report suspicious activity on my account immediately?", got tagged Urgent_Support with 86% confidence, even though it is really a how-to question. That is the model picking up on words like "immediately" and "suspicious" without understanding that the sentence is asking for instructions, not reporting an active emergency.

After refinement, Table III shows the before-and-after for the seven phrases I re-tested. Three flipped to the correct label, four stayed wrong, and none of them regressed from correct to incorrect. That is a modest improvement, not a fix, and I think that gap is worth discussing on its own, which I do in Section IV-B.

## IV. DISCUSSION

### A. Explicit Rules vs. Tagged Learning

If I tried to handle this with IF/ELSE rules instead of a trained model, the first problem I would run into is that there is no fixed list of complaint words. My Negative_Complaint examples alone use words like "crashing," "confusing," "frustrating," "ripped," "froze," and "disappeared," and that is just a fraction of the ways a fan can describe something going wrong. Writing a rule for each one would take forever, and I would still miss the next phrasing someone comes up with tomorrow.

The bigger problem shows up in Test B. A rule like "if the sentence contains a positive word, tag it Positive_Praise" breaks immediately on "Oh great, another restock that sold out before the page even finished loading." The word "great" is there, but the sentence is a complaint. No simple keyword rule can tell the difference, because the meaning depends on the whole sentence, not one word in isolation. A trained model at least has a shot at learning that pattern if it sees enough examples of it, which is exactly what my refinement step tried to do. Rules cannot learn from examples the way a network can; someone has to sit down and write every exception by hand, and sarcasm is basically an infinite list of exceptions.

So the real challenge is not that rules are hard to write, it is that customer language does not sit still. New slang, new sarcasm, new ways of complaining about a broken checkout keep showing up, and a rule-based system needs a human to notice each one and patch it in. A tagged model can pick some of that up automatically as long as it sees representative examples, which is the whole point of Step 5 in this exercise.

### B. Data Bias and Dataset Quality

The exercise asks what happens with 3 examples per class versus 10, and I can extend that comparison using my own numbers, since I went from 100 examples per class up to 100 plus roughly 32 refinement examples. Even with 100 examples per class already in place, adding around 32 more, targeted specifically at the failure patterns from Test B, only fixed 3 of 7 retested phrases. That tells me two things.

First, volume alone is not the fix. I already had 100 examples per class before refinement, so the problem in Test B was not that the model lacked data in general, it lacked a specific kind of data: sarcasm, sentiment reversal, and hedged urgency. Adding more plain praise or plain complaints would not have helped, because the model was never confused about plain praise or plain complaints in the first place. This is what I take "representative diversity" to mean in the exercise's third question. It is not just about having a lot of examples, it is about having examples that cover the actual range of ways people write, including the tricky ways.

Second, targeted data helps, but it does not generalize perfectly. Three of seven retested phrases flipped to correct, which is real progress, but four stayed wrong even after I added examples clearly built around the same pattern. My guess is that a phrase like "This is the first ticket drop that didn't crash the app, genuinely impressed" uses a slightly different structure than the training examples I added, so the model did not connect it to the new pattern. With a small dataset, like the 3-examples-per-class case the exercise mentions, I would expect this problem to be much worse, since the model would have almost nothing to generalize from and would likely just memorize a handful of exact phrases instead of learning the underlying pattern. That is really the core of why deep learning accuracy depends so heavily on data quality, volume, and diversity together. Volume without diversity gives you a model that is confident on the phrasing it has seen and lost on everything else, and diversity without enough volume per pattern gives you a model that half-learns a pattern and cannot apply it consistently.

### *C. Connecting to the Slide Architecture: Where Feature Extraction Happens*

Based on how the tool behaves, I think feature extraction happens right at the start of the "Learn & Test" stage, before the actual neural network training runs. When I clicked "Train (New Machine Learning Model)," the tool has to first turn every one of my 300 sentences into some numerical form the network can actually do math on, since a neural network cannot operate on raw words. That conversion step, turning text into numbers, is what I would call feature extraction here.

My best estimate is that this step works on individual words or short word combinations rather than whole sentences, and it is a bag-of-words or similar type of representation rather than something that tracks word order carefully. I say this because of how Test B failed. If the model were extracting features that captured full sentence structure and word order, it would have picked up on the "but" in "I was ready to cancel my membership, but this renewal experience actually changed my mind" as a signal that the sentiment flips partway through. Instead, it looks like the model leaned on the presence of words like "cancel" and treated the sentence as negative overall, missing the reversal. That is consistent with a feature extraction step that counts or weighs words without fully modeling their order and relationship to each other.

So in terms of the pipeline, I would place feature extraction between Step 2 (tagging the training examples) and the actual weight-updating part of Step 3. The tagging step gives the network raw labeled text. Feature extraction is the translation layer that turns that raw text into numbers. Only after that translation can the network start finding mathematical associations between those numbers and the three labels, which is the part the exercise describes as building the classifier without hand-written IF/THEN rules.

## V. Conclusion

Building this classifier made the tradeoff between rules and tagged learning very concrete for me. Rules fail because customer language is too varied to list out by hand, and tagged learning fails in its own way when the training data does not cover a pattern, like sarcasm or hedged urgency, even if the overall dataset is large. My 90% accuracy on clear phrasing versus 33% on ambiguous phrasing shows the same model can be strong and weak at the same time, depending on what you ask of it. The refinement round backs this up: adding 32 targeted examples fixed under half of the failures I retested, which tells me that fixing a blind spot in a model takes more than a handful of new examples, it usually takes several rounds of testing, spotting the pattern, and adding more varied examples until the model generalizes rather than memorizes. That iterative loop, test, find the gap, add better data, retrain, is really the main lesson I am taking from this exercise.